

CAN YOU BUILD THIS YOURSELF? — HONEST ASSESSMENT & LEARNING PATH

The Brutal Truth First

Building this entire system from zero programming knowledge is not realistic in under 12-18 months.

This is not a “learn to code in 3 months” project. It is a production-grade fintech SaaS with:

- Database design and SQL queries
- Async Python backend (FastAPI)
- Frontend framework (React + TypeScript)
- Docker containerization
- Bank API integrations (PSD2 / Open Banking)
- Security compliance (GDPR, PCI-DSS)
- DevOps and cloud deployment

Each of those alone takes 3-6 months to learn properly. Combined, you are looking at **1.5-2 years of full-time study** before you could build and deploy something production-ready without help.

However — and this is important — there is a middle path that makes much more sense for you.

The Smart Hybrid Approach (Recommended)

Instead of trying to become a senior developer, become a **technical founder** who can:

1. **Understand** the codebase enough to review work and catch issues
2. **Manage** a programmer effectively with precise requirements
3. **Modify** small things (text, colors, simple logic) without calling your dev
4. **Debug** basic problems before escalating
5. **Eventually** handle 20-30% of ongoing development yourself

This is achievable in **6-12 months** of consistent study (5-10 hours/week).

What You Should Do RIGHT NOW (Week 1)

Option A: You have a programmer starting soon

Do NOT try to learn to code before they arrive. Instead:

- Read the Implementation Roadmap I gave you — understand the phases
- Learn what Docker is (30-minute YouTube video)
- Learn what an API is (30-minute YouTube video)
- Set up a free GitHub account

- Watch your programmer set up the project — ask them to explain each file
- Install VS Code on your computer so you can open and read the code

Goal by end of Week 1: You can open the project folder, recognize what files do what, and run `docker-compose` up without help.

Option B: You don't have a programmer yet

Hire one first. Do not delay launch to learn coding. While you search:

- Do the same Week 1 tasks above
- Read “The Lean Startup” by Eric Ries (understand MVP philosophy)
- Write your user stories: “As a merchant, I want to upload my daily collection CSV so that I can see what cleared”

6-Month Learning Path: From Zero to “Technical Founder”

Month 1: Foundations (No Code Yet)

Goal: Understand how software works conceptually

Week	Topic	Time	Resource
1	How the internet works (HTTP, DNS, servers)	3h	freeCodeCamp YouTube
1	What is a database? (tables, rows, queries)	3h	SQLBolt.com (free)
2	What is an API? (REST, JSON, endpoints)	3h	Postman “API 101” (free)
2	Git basics (commit, push, pull, branch)	4h	GitHub Learning Lab (free)
3	Docker basics (containers, images, compose)	4h	Docker “Getting Started” (free)
3	Read YOUR codebase — just read, don't write	5h	Ask your programmer to walk you through
4	Terminal/command line basics	3h	Codecademy “Learn the Command Line”

End of Month 1: You can explain to someone how your app works from browser → API → database → back. You can read code and roughly understand what each file does.

Month 2: Python Basics

Goal: Write simple scripts and understand your backend code

Week	Topic	Time	Resource
5-6	Python syntax (variables, loops, functions, classes)	10h	“Automate the Boring Stuff with Python” (free online)
7	Python for data: lists, dictionaries, CSV parsing	5h	Same book, chapters on file handling
7	Decimal and date handling (critical for money!)	3h	Python docs + your codebase examples
8	Read your <code>models_orm.py</code> file line by line	5h	Ask your programmer to explain each line

End of Month 2: You can write a Python script that reads a CSV and calculates totals. You understand what SQLAlchemy models represent.

Month 3: Databases & SQL

Goal: Query your own database and understand the schema

Week	Topic	Time	Resource
9-10	SQL fundamentals (SELECT, WHERE, JOIN, GROUP BY)	10h	SQLBolt.com + Mode Analytics SQL Tutorial
10	PostgreSQL specifics (psql CLI, pgAdmin GUI)	4h	Official PostgreSQL tutorial
11	Query YOUR database: “Show me all uncleared collections”	5h	Write queries against your dev database
11	Database design principles (normalization, indexes)	3h	Read your ERD diagram; ask programmer why each index exists
12	Alembic migrations (what they are, how they work)	3h	Your programmer shows you one migration

End of Month 3: You can open pgAdmin, run queries on your database, and understand why the schema is designed the way it is.

Month 4: Backend Deep Dive

Goal: Understand FastAPI and modify simple endpoints

Week	Topic	Time	Resource
13-14	FastAPI tutorial (official docs)	10h	fastapi.tiangolo.com/tutorial
14	Pydantic models (data validation)	4h	FastAPI tutorial covers this
15	Async/await in Python (why your code uses <code>async</code>)	4h	RealPython “Async IO in Python”
15	Read your <code>main.py</code> and one router file completely	5h	Trace a request: <code>POST /collections</code> → database → response
16	Try modifying one endpoint (e.g., add a new filter)	5h	With your programmer reviewing

End of Month 4: You can add a simple query parameter to an existing endpoint. You understand the request/response flow.

Month 5: Frontend Basics

Goal: Understand React and modify the UI

Week	Topic	Time	Resource
17-18	HTML & CSS fundamentals	8h	freeCodeCamp Responsive Web Design
18	JavaScript basics (not TypeScript yet)	6h	JavaScript.info (free)
19	React fundamentals (components, props, state)	10h	React official tutorial (react.dev)
19	TypeScript basics (types, interfaces)	4h	TypeScript Handbook (official)
20	Read your frontend codebase	5h	Identify which component renders the dashboard

End of Month 5: You can change text, colors, and simple layout in your React app. You understand component structure.

Month 6: Integration & DevOps

Goal: Deploy changes and understand the full pipeline

Week	Topic	Time	Resource
21	Git workflow (branch, PR, merge, resolve conflicts)	4h	Practice with your programmer
21	GitHub Actions (what CI/CD does)	3h	Read your <code>.github/workflows/ci.yml</code>
22	Docker Compose deep dive	4h	Official docs + your <code>docker-compose.yml</code>
22	Environment variables and secrets	2h	Your <code>.env.example</code> file
23	AWS basics (EC2, RDS, S3 concepts)	5h	AWS Free Tier tutorials
23	Make your first end-to-end change	6h	Change something in frontend → backend → database
24	Deploy your change to staging	4h	With programmer supervision

End of Month 6: You can make a small change (e.g., add a new field to a form), test it locally, push to GitHub, and deploy to staging.

What You Will NOT Be Able to Do After 6 Months

Be realistic. After 6 months, you will NOT be able to:

- ❌ Write the reconciliation matching engine from scratch
- ❌ Design secure authentication systems
- ❌ Build bank API integrations (PSD2 / Open Banking)
- ❌ Optimize database queries for performance
- ❌ Handle security vulnerabilities
- ❌ Set up production infrastructure alone
- ❌ Write automated tests
- ❌ Review code for architectural problems

These require 3-5 years of experience. That's why you hire a programmer.

What You WILL Be Able to Do After 6 Months

You WILL be able to:

- ✅ Explain your tech stack to investors and partners
- ✅ Read pull requests and ask intelligent questions
- ✅ Fix a typo in an email template without bothering your dev

- ☒ Add a new column to a report (with guidance)
 - ☒ Query the database to answer business questions
 - ☒ Understand why a bug happened and describe it precisely
 - ☒ Estimate whether a feature request is “small” or “large”
 - ☒ Onboard a new programmer by showing them the codebase
 - ☒ Not get fooled by a bad programmer or inflated estimates
-

The “Learn to Code” Trap

Many non-technical founders fall into this trap:

1. They try to learn everything themselves
2. They spend 6 months learning, make slow progress
3. Their competitor launches while they’re still on Chapter 3 of a Python book
4. They burn out and abandon the project

Don’t be that person.

The winning strategy:

1. **Hire a programmer NOW** (or find a technical co-founder)
 2. **Learn in parallel** — 5-10 hours/week, not 40 hours/week
 3. **Use your learning to be a better manager**, not to replace your programmer
 4. **Gradually take on more** as you get comfortable (Month 6-12)
 5. **Eventually** (Year 2-3), you might handle 30-40% of development yourself
-

If You Are SERIOUS About Learning to Code

Full-time path (40+ hours/week):

- Month 1-3: Python + SQL + Git (foundations)
- Month 4-6: FastAPI + PostgreSQL + Docker (backend)
- Month 7-9: React + TypeScript + CSS (frontend)
- Month 10-12: Build a simplified version of your app (no bank APIs, no auth, local only)
- Month 13-18: Add real features, security, deployment
- Month 19-24: Production readiness, testing, scaling

Realistic timeline to solo-build this system: 18-24 months.

Cost:

- Your time: 2,000+ hours
- Lost opportunity: 18-24 months of no product, no customers, no revenue

- Risk: You might discover you don't enjoy coding and quit after 6 months
-

My Strong Recommendation

Do **BOTH** simultaneously:

Timeline	You (Learning)	Programmer (Building)
Month 1-3	Learn foundations (5h/week)	Builds MVP (Phase 1-4)
Month 4-6	Learn to read/modify code (5h/week)	Builds integrations (Phase 5-7)
Month 7-9	Handle small frontend changes	Finishes testing & launch (Phase 8-10)
Month 10-12	Handle 20% of ongoing dev	Handles 80% + mentors you
Year 2	Handle 30-40% of dev	Focuses on complex features
Year 3	Handle 50% of dev	Becomes CTO / advisor

This way:

- Your product launches in 6 months, not 24
 - You learn by working on real code, not toy examples
 - Your programmer teaches you (free mentoring)
 - You gradually reduce dependency on your programmer
 - You save €30,000-€50,000 in dev costs over 2 years
-

Free Resources to Start TODAY

Python:

- “Automate the Boring Stuff with Python” — al Sweigart (free at automatetheboringstuff.com)
- Python.org official tutorial

SQL:

- SQLBolt.com (interactive, free)
- Mode Analytics SQL Tutorial (free)

Web Development:

- freeCodeCamp.org (free, comprehensive)
- The Odin Project (free, project-based)

Your Specific Stack:

- FastAPI tutorial: fastapi.tiangolo.com/tutorial
- React tutorial: react.dev/learn
- Docker tutorial: docs.docker.com/get-started

YouTube Channels:

- Fireship (10-minute overviews of technologies)
 - Traversy Media (project-based tutorials)
 - freeCodeCamp (long-form courses)
-

Final Word

You do not need to become a programmer to build a successful tech company.

Elon Musk does not write the Tesla autopilot code. **Jeff Bezos** did not build the Amazon checkout system. **Brian Chesky** (Airbnb) was a designer, not a programmer, when he started.

They hired great engineers and learned enough to lead them effectively.

Your job is to:

- Know the customer (you already do — you live this problem)
- Define the product vision
- Sell and market
- Manage the business
- Learn enough tech to be dangerous

Your programmer's job is to:

- Build the system
- Make technical decisions
- Ensure security and reliability
- Teach you along the way

That division of labor is how successful startups are built.

If you want me to create a personalized weekly study plan based on how many hours per week you can commit, tell me your availability and I'll build it.